# แบบฝึกหัดปฏิบัติการคาบที่ 2: Class and Object

2. จงหาข้อผิดพลาดของส่วนของโปรแกรมต่อไปนี้ โดยวงกลมตำแหน่งที่ผิด เขียนหมายเลขกำกับแต่ละตำแหน่งและอธิบายเหตุผลด้านล่างว่าเหตุใดตำแหน่งดังกล่าวจึงผิด ถ้าไม่มีข้อผิดพลาดให้ตอบว่าไม่มีข้อผิดพลาด

| ส่วนของโปรแกรม | ผลลัพธ์ |
|---|---|
| `public class ShowErrors{`<br>`  public static void main(String[] args){`<br>`  ShowErrors t= new ShowErrors(5);`<br>`  }`<br>`}` | class นี้ไม่มี constructor ที่จะรับตัวแปรประเภทตัวเลข |
| `public class ShowErrors{`<br>`  public static void main(String[] args){`<br>`  ShowErrors t= new ShowErrors();`<br>`  t.x();`<br>`  }`<br>`}` | class นี้ไม่มี method x<br>แต่ถ้าหากว่ามี method x นิยามอยู่ที่อื่น โปรแกรมนี้ไม่มีข้อผิดพลาด (แต่ก็ไม่ได้อยู่ดี เพราะว่า class ShowErrors ถูกนิยามปิดด้วย {} ครบหมดแล้ว) |
| `public class ShowErrors{`<br>`  public void method1(){`<br>`  Circle c;`<br>`  System.out.println("What is radius "+ c.getRadius());`<br>`  c=new Circle;`<br>`  }`<br>`}` | ไม่สามารถเรียก getRadius() ผ่านตัวแปร c ได้ เพราะ c ยังไม่ได้อ้างอิง object<br>และ บรรทัดต่อมาก็ผิด เพราะไม่มีวงเล็บหลัง Circle |
| `public class ShowErrors{`<br>`  public static void main(String[] args){`<br>`  C c = new C(5.0);`<br>`  System.out.println(c.value);`<br>`  }`<br>`}`<br>`class C{`<br>`  int value=2;`<br>`}` | class C ไม่มี constructor<br>แต่สามารถ print ได้ ถ้าหากว่านำ 5.0 ออกจาก constructor ใน new |

3. จงอธิบายการทำงานของโปรแกรมต่อไปนี้

| โปรแกรม | แสดงผลลัพธ์จากโปรแกรม |
|---|---|
| `public class Test{`<br>`  public static void main(String[] args){`<br>`  Count myCount = new Count();`<br>`  int times=0;`<br>`  for (int i=0; i<100 ;i++)`<br>`  increment(myCount, times);`<br>`  System.out.println("count is "+myCount.count);`<br>`  System.out.println("times is "+times);`<br>`  }`<br>`  public static void increment(Count c, int times){`<br>`  c.count++;`<br>`  times++;`<br>`  }`<br>`}`<br>`public class Count{`<br>`  public int count;`<br><br>`  public Count(int c){`<br>`  count =c;`<br>`  }`<br>`  public Count(){`<br>`  count =1;`<br>`  }`<br>`}` | `count is 101`<br>`times is 0` |

3.1 ข้อแตกต่างของการส่งค่าพารามิเตอร์ของ Primitive type และการส่งค่าพารามิเตอร์ของ reference type จากโปรแกรมต่อไปนี้

แบบ primitive type คือการส่ง***ค่า***ของตัวแปรนั้นไปให้ method เพื่อใช้เฉย ๆ แต่แบบ reference type คือการส่ง***ที่อยู่***ของค่านั้น ๆ ในที่นี้ ที่อยู่คือ object ส่งผลให้ตอนเกิดการแก้ไขค่า แบบ primitive type จะไม่เกิดการแก้อะไรทั้งสิ้น แต่แบบ reference type จะมีการแก้ค่าต้นฉบับใน object เกิดขึ้น

3.2 ระบุจุดที่มีการส่งค่าพารามิเตอร์ของ Primitive type และการส่งค่าพารามิเตอร์ของ reference type

| | |
|---|---|
| `Count myCount = new Count();`<br>`int times=0;`<br>`for (int i=0; i<100 ;i++)`<br>`  increment(myCount, times);` | myCount คือ reference type โดยค่าด้านใน myCount จะถูกอ้างอิงจาก myCount<br>times คือ primitive type |

4. จงแสดง output จากโปรแกรมต่อไปนี้พร้อมอธิบายการทำงานในแต่ละบรรทัด

| ส่วนของโปรแกรม | ผลลัพธ์ |
|---|---|
| `public class Test {`<br>`  public static void main(String[] args) {`<br>`  int[] a = {1, 2}; // ประกาศอาเรย์ 1 มิติของตัวเลข`<br>`  swap(a[0], a[1]); // เรียกใช้ฟังก์ชัน โดยโยนค่าของ a[0], a[1] เข้าไป`<br>`  System.out.println("a[0] = " + a[0]`<br>`  + " a[1] = " + a[1]);`<br>`  }`<br>`  public static void swap(int n1, int n2) {`<br>`  int temp = n1; // ประกาศตัวแปรชั่วคราว ให้มีค่าเป็น n1 (ซึ่งเป็นค่าที่รับมา)`<br>`  n1 = n2; // ให้ n1 มีค่าเป็น n2 (ซึ่งเป็นค่าที่รับมา)`<br>`  n2 = temp; // ให้ n2 มีค่าเป็น temp`<br>`  }`<br>`}` | `a[0] = 1 a[1] = 2` |
| `public class Test {`<br>`  public static void main(String[] args) {`<br>`  T t1 = new T(); // สร้าง object T ใหม่ เป็น t1`<br>`  T t2 = new T(); // สร้าง object T ใหม่ เป็น t2`<br>`  System.out.println("t1's i = " + t1.i + " and j = " + t1.j);`<br>`  System.out.println("t2's i = " + t2.i + " and j = " + t2.j);`<br>`  }`<br>`}`<br>`class T {`<br>`  static int i = 0; // เป็นตัวแปรของ class นี้โดยเฉพาะ (static), i นี้จะไม่เป็นของ object T เมื่อถูกสร้าง แต่เป็นของ class T, หรือก็คือ ทุก object T จะมีตัวแปร i นี้เท่ากันหมดทุกตัว เสมอไป`<br>`  int j = 0; // เป็นตัวแปรตัวเลขของ object`<br>`  T() { // constructor แห่ง object`<br>`  i++;`<br>`  j = 1;`<br>`  }`<br>`}` | `t1's i = 2 and j = 1`<br>`t2's i = 2 and j = 1` |

8. *(Geometry: n-sided regular polygon)* In an n-sided regular polygon, all sides have the same length and all angles have the same degree (i.e., the polygon is both equilateral and equiangular). Design a class named RegularPolygon that contains:

- A private **`int`** data field named **`n`** that defines the number of sides in the polygon with default value **`3`**.
- A private **`double`** data field named **`side`** that stores the length of the side with default value **`1`**.
- A private **`double`** data field named **`x`** that defines the x-coordinate of the polygon's center with default value **`0`**.
- A private double data field named **`y`** that defines the y-coordinate of the polygon's center with default value **`0`**.
- A no-arg constructor that creates a regular polygon with default values.
- A constructor that creates a regular polygon with the specified number of sides and length of side, centered at (**`0`**, **`0`**).
- A constructor that creates a regular polygon with the specified number of sides, length of side, and (x,y) coordinates.
- The accessor and mutator methods for all data fields.
- The method **`getPerimeter()`** that returns the perimeter of the polygon.
- The method **`getArea()`** that returns the area of the polygon. The formula computing the area of a regular polygon is $\text{area} = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$

Draw the UML diagram for the class and then implement the class. Write a test program that creates three **`RegularPolygon`** objects, created using the no-arg constructor, using **`RegularPolygon(6, 4)`**, and using **`RegularPolygon(10, 4, 5.6, 7.8)`**. For each object, display its perimeter and area.

